

Parking Lot Occupancy Detection

Complete Guide: Architecture, Code, and Reuse for Any Parking Lot

1. System Architecture

This system detects whether individual parking spots are **empty** or **occupied** using CCTV footage. It uses a **binary mask** to locate spots, **connected components** to extract bounding boxes, a **trained SVM classifier** to predict status, and **movement detection** to optimize performance. The mask is the bridge between training images and video frames: same coordinates crop both.

Stage	Component	Purpose
1. Input	Binary mask (.png)	White regions = parking spots
2. Detection	connectedComponentsWithStats()	Label each spot, extract bounding boxes
3. Movement	calc_diff()	Compare frames, find spots with changes
4. Classify	empty_or_not()	SVM model: empty (True) or occupied (False)
5. Display	cv2.rectangle() + putText()	Green=empty, Red=occupied + available count

2. The Mask

A binary image (same resolution as video) where white = parking spot, black = everything else. Without it, you classify the entire frame with no way to identify individual spots. **You need a NEW mask for each parking lot** (different layout/camera angle), but reuse the same mask across different times, weather, and cars for the same lot.

3. Training Workflow

The same workflow applies to **every parking system project**. Only the data changes:

Step	Action	Details
1	Data Collection	Get CCTV footage or use existing dataset
2	Extract & Label	Crop spots using mask, label as empty / not_empty
3	Split Data	80% training, 20% validation
4	Train	Resize to 15x15, flatten, train SVM classifier
5	Validate	Test on unseen data, target 85%+ accuracy
6	Save	pickle.dump(model, open('model.p', 'wb'))

Key: You do NOT classify every frame during training. Take a few frames, crop each spot, label manually, and train. Training data = individual spot images, not full frames.

4. Inference Pipeline (main.py)

Real-world insight: It takes 2-5 seconds to park a car, so classifying every frame is wasteful. Most spots are in steady state for hours (people shop or work). We only classify spots where **movement is detected**, every 2-3 seconds.

```
import cv2
import numpy as np
from util import get_parking_spots_bboxes, empty_or_not

mask_path = r'D:\Computer Vision\Parking System\parking\mask_1920_1080.png'
video_path = r'D:\Computer Vision\Parking System\parking\parking_1920_1080.mp4'

def calc_diff(im1, im2):
    '''Average pixel intensity difference between two spot images.'''
    return np.abs(np.mean(im1) - np.mean(im2))

mask = cv2.imread(mask_path, 0) # Load binary mask (grayscale)
cap = cv2.VideoCapture(video_path) # Open video
connected_components = cv2.connectedComponentsWithStats(mask, 4, cv2.CV_32S)
spots = get_parking_spots_bboxes(connected_components) # Get [x,y,w,h] per spot

spots_status = [None for j in spots] # True=empty, False=occupied
diffs = [None for j in spots] # Movement diff per spot
previous_frame = None
frame_nmr = 0
ret = True
step = 90 # Classify every ~3 sec at 30 FPS

while ret:
    ret, frame = cap.read()

    # STEP 1: Calculate movement diffs
    if frame_nmr % step == 0 and previous_frame is not None:
        for spot_index, spot in enumerate(spots):
            x1, y1, w, h = spot
            spot_crop = frame[y1:y1+h, x1:x1+w, :]
            diffs[spot_index] = calc_diff(spot_crop,
                                           previous_frame[y1:y1+h, x1:x1+w, :])

    # STEP 2: Classify only high-movement spots
    if frame_nmr % step == 0:
        if previous_frame is None:
            arr_ = range(len(spots)) # First frame: all
        else:
            arr_ = [j for j in range(len(spots))
                    if diffs[j] / np.amax(diffs) > 0.4]
        for spot_index in arr_:
            x1, y1, w, h = spots[spot_index]
            spot_crop = frame[y1:y1+h, x1:x1+w, :]
            status = empty_or_not(spot_crop)
            spots_status[spot_index] = status

    if frame_nmr % step == 0:
        previous_frame = frame.copy()

    # STEP 3: Draw colored rectangles
    for spot_index, spot in enumerate(spots):
        x1, y1, w, h = spot
        status = spots_status[spot_index]
        if status is None:
            cv2.rectangle(frame, (x1, y1), (x1+w, y1+h), (128, 128, 128), 2)
        elif status:
            cv2.rectangle(frame, (x1, y1), (x1+w, y1+h), (0, 255, 0), 2)
        else:
            cv2.rectangle(frame, (x1, y1), (x1+w, y1+h), (0, 0, 255), 2)

    # STEP 4: Show available count
    cv2.rectangle(frame, (80, 20), (550, 80), (0, 0, 0), -1)
    cv2.putText(frame, f'Available: {sum(spots_status)}/{len(spots_status)}',
```

```
        (100,60), cv2.FONT_HERSHEY_SIMPLEX, 1, (255,255,255), 2)

cv2.namedWindow('Parking Video', cv2.WINDOW_NORMAL)
cv2.imshow('Parking Video', frame)
if cv2.waitKey(25) & 0xFF == ord('q'): break
frame_nmr += 1

cap.release()
cv2.destroyAllWindows()
```

5. Movement Detection & Threshold

calc_diff() compares average pixel intensity of a spot between two frames. Low diff (0-1) = static spot (skip). High diff (5+) = car entering/leaving (classify). The **relative threshold** normalizes each diff by the max diff, then classifies only spots above 0.4 (top 40% of movement).

```
# Example: 8 spots
diffs      = [0.2, 1.5, 3.2, 0.1, 4.5, 2.1, 0.3, 5.0]
max_diff   = 5.0
ratios     = [0.04, 0.30, 0.64, 0.02, 0.90, 0.42, 0.06, 1.00]
above 0.4 = [          Spot2,          Spot4, Spot5,          Spot7]
# Only classify 4 out of 8 spots = 50% CPU saved!
```

Plotting for Threshold Selection

Never guess the threshold. Plot a histogram of all diff values from the video. You will see two groups: static spots (left peak) and moving spots (right tail). The **gap between the two groups** is your optimal threshold. This is data-driven decision making, not trial-and-error.

```
import matplotlib.pyplot as plt
plt.hist(all_diffs, bins=50, edgecolor='black')
plt.axvline(x=0.3, color='r', linestyle='--', label='Threshold')
plt.xlabel('Diff Value'); plt.ylabel('Frequency'); plt.legend(); plt.show()
```

Step Value	Interval	Use Case
step = 30	~1 sec	Fast updates, may flicker
step = 60	~2 sec	Less flicker, slight delay
step = 90	~3 sec	Best balance (recommended)
step = 150	~5 sec	Smoothest but too slow

6. Utility Functions (util.py)

```
import pickle, warnings
from skimage.transform import resize
import numpy as np, cv2

EMPTY, NOT_EMPTY = True, False
warnings.filterwarnings('ignore', category=UserWarning)
MODEL = pickle.load(open('model.p', 'rb'))

def empty_or_not(spot_bgr):
    '''Classify one spot: True=empty, False=occupied'''
    img_resized = resize(spot_bgr, (15, 15, 3))
    flat_data = np.array([img_resized.flatten()])
    return EMPTY if MODEL.predict(flat_data) == 0 else NOT_EMPTY

def get_parking_spots_bboxes(connected_components):
    '''Extract [x, y, w, h] bounding boxes from mask.'''
    (totalLabels, label_ids, values, centroid) = connected_components
    slots = []
    for i in range(1, totalLabels): # Skip background (label 0)
        x1 = int(values[i, cv2.CC_STAT_LEFT])
        y1 = int(values[i, cv2.CC_STAT_TOP])
        w = int(values[i, cv2.CC_STAT_WIDTH])
        h = int(values[i, cv2.CC_STAT_HEIGHT])
        slots.append([x1, y1, w, h])
    return slots
```

7. Reusing for a New Parking Lot

This is a **very modular project**. To deploy on any new parking lot, you just need CCTV footage. The pipeline and code stay the same. Only the data and mask change.

Step	Action	Time
1	Get CCTV footage of the new parking lot	Available
2	Extract one reference frame from the video	1 min
3	Create a NEW binary mask (draw spots on frame)	15-30 min
4	Verify mask resolution = video resolution	1 min
5	Extract frames, crop spots, label empty/not_empty	30-60 min
6	Train new classifier, validate accuracy	5-10 min
7	Save as model.p, update paths in main.py	2 min
8	Run, plot diffs histogram, adjust threshold	10 min

What Changes vs What Stays

Component	Changes?	Details
main.py	Minimal	Only update mask_path and video_path
util.py	NO	Same functions for any parking lot
mask.png	YES	New mask for each lot / camera angle
model.p	YES	Retrain on new lot's data
clf-data/	YES	New spot images from new lot
Pipeline logic	NO	Same loop, same movement detection
Threshold	MAYBE	Plot histogram and adjust

8. Key Lessons

Understand the domain: Parking lots are mostly static. Cars park for hours. Classifying every frame is wasteful. Classify every 2-3 seconds.

Mask = bridge: Same coordinates crop training images and video frames. Without it, no per-spot classification.

Match resolutions: Wrong mask + video combo (e.g., mask_crop with full video) causes crashes.

Plot before picking threshold: Never guess. Histogram shows the natural gap between static and moving spots.

Relative threshold: $\text{diffs}[]/\text{max_diff} > 0.4$ adapts to different lighting automatically.

previous_frame is None: Always check before comparing. First frame has nothing to compare against.

Modular design: Swap data + mask, retrain model, everything else works. No code rewrite needed.

Dependencies: opencv-python, numpy, scikit-image, scikit-learn, matplotlib
